ENERGY AND CARBON DISCLOSURE · OFFICIAL · PUBLIC

What our software consumed, April to June 2026

Calendar quarter, 1 April to 30 June 2026, 90 days. Measured by Perf Sentinel 0.7.0.

Energy, operational	Carbon, location-based	Avoidable with known fixes
17.31 kWh	1.80 $kgCO_2eq$	0.32 $kgCO_2eq$
4 150 000 requests measured, 98.8 % of those declared	a directional estimate, true value likely between 0.90 and 3.60	18 % of measured I/O, at the fixed canonical threshold

● DECLARED: HASH ONLY, NOT SIGNED

This document carries the machine-readable report it was rendered from, as an attachment. The printed code alone proves nothing: the proof is the attached report, rechecked with the command on page 5. This report is not signed.

document code
sha256: 4565 7ef6 79b1 aa22 7500 f72c f548 23b2 bbc8 ad86 f0d9 59f7 3a29 1cd2 d52a 7953

AT A GLANCE

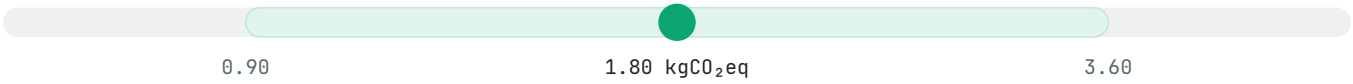
Three numbers, and how sure we are of them

Energy is measured from the work the services did. Carbon applies the carbon intensity of the electricity grid to that energy. Avoidable is what a known class of defects, repeated database and HTTP calls, wasted.

Energy, operational	Carbon, location-based	Efficiency score
17.31 kWh	1.80 $kgCO_2eq$	82.1 / 100
over 90 days, 4 150 000 requests	a directional estimate, see the band below	higher is better, computed by the engine

The carbon figure and its uncertainty

The method is a directional estimate with a ~2x bracket. The true value is likely to sit anywhere in the shaded band.



Avoidable waste, two readings of the same data



The canonical figure uses a threshold fixed in the measuring binary, so the operator cannot shrink it by tuning. The operational figure uses the operator's own threshold and is what their dashboard shows. Both are published so a reader can compare.

How much of the quarter was actually measured



Runtime coverage 91 %

164 of 180 scoring windows carried per-service energy attribution. The other 16 fell back to a proportional share of I/O operations. An official report needs at least 75 %.

Temporal coverage 96.7 %

87 of 90 days carried measured traffic, the largest gap was 2 days. Quiet days lower this figure too, so it is a floor on activity, not an uptime claim.

Where these numbers go in European climate reporting

Not reported

Which services consumed what

Three of the four applications Example SAS declared were measured. Per-service carbon is operational only. The count of anti-patterns is published, their detail is not, by choice of granularity.

Application	Requests	Share of energy	kWh	$kgCO_2eq$	Efficiency	Anti-patterns
Checkout	2 100 000	51.1 %	8.84	0.92	78.0	20
Catalog	1 500 000	36.4 %	6.30	0.66	86.0	22
Billing	550 000	12.6 %	2.18	0.22	84.5	5
legacy-batch	Excluded by the operator: OTel instrumentation pending, scheduled Q2. Declared, not measured.					
Total	4 150 000		17.31	1.80	82.1	47

Scope, as declared and as measured

Declared by the operator	4 applications · 1 environment (prod) · 4 200 000 requests in the period
Derived by the engine	3 applications measured · 4 150 000 requests measured · 98.8 % coverage
Region	eu-west-3 · carbon intensity from electricity_maps

Reading the efficiency score

The score rewards services whose I/O is useful work rather than repetition. A lower score does not mean a larger footprint: a service can score higher than another while consuming less, because it carries less traffic, not because it is better written.

How the figures were produced, and what they are not

Specification	ISO/IEC 21031:2024 · conformance core-required
Measuring binary	perf-sentinel 0.7.0 · SPECpower table 2026-04-24 (CCF aligned) · energy models cloud_specpower, io_proxy_v3
Detectors enabled	n_plus_one_sql, n_plus_one_http, redundant_sql, redundant_http (core, required) · slow_sql, slow_http, excessive_fanout, chatty_service, pool_saturation, serialized_calls
Detectors disabled	none
Runtime calibration	not applied this period · 164 runtime windows, 16 fallback windows
Archive	Not reported

Disclaimers, exactly as the report carries them

1. Directional estimate, not regulatory-grade.
2. Approximate uncertainty bracket: ~2x multiplicative.
3. Not suitable for CSRD or GHG Protocol Scope 3 reporting.
4. Methodology: ISO/IEC 21031:2024 (SCI).
5. Per-service carbon includes operational emissions only; embodied carbon (SCI M term) is reported in the aggregate total but not attributed per service.
6. Energy and carbon attribution per service is runtime-calibrated when the window's energy_model is non-empty; archives without runtime energy attribution fall back to proportional I/O share.
7. Temporal coverage: 87/90 declared days carried measured traffic (96.7%, largest gap 2 days). Archiving is traffic-gated, so days without measured traffic, including legitimately quiet periods, lower this figure. It is a lower bound on activity, not a daemon-uptime guarantee.

What this document is

A public transparency report produced by an open-source tool, published by Example SAS on its own responsibility. It is not an audited inventory and does not make a CSRD submission. The scope is IT compute observed through application traces: no end-user devices, no buildings.

VERIFY THIS DOCUMENT

Five minutes to know it was not altered

The check is on the report attached inside this PDF, not on its layout.

Claim	Checked by
The figures match document code sha256: 4565 7ef6 79b1 aa22 7500 f72c f548 23b2 bbc8 ad86 f0d9 59f7 3a29 1cd2 d52a 7953	perf-sentinel when this PDF was rendered, and you
The signed attestation names that same code	perf-sentinel when this PDF was rendered, and you
Example SAS's pipeline signed it, in the public Sigstore log	nobody, this report is not signed
The measuring binary is an official Perf Sentinel release	you, with the command below
An auditor reviewed the figures	nobody, this report is not audited

Attached to this PDF

- ☐ **report.json** · source — 5 KB
  `sha256 88073b39a08177b914b466dfffa89707645a3eef381f9a392851f3d1474a2c8b`
- ☐ **attestation.intoto.jsonl** · supplement, names the document code — 1 KB
  `sha256 9e065457912f1a64537447d0a5f90c2335b14dd3175c1e72aa181382350086b0`

If you do not run commands

1. Ask Example SAS for the page where it publishes its reports. It should list the document code of each report.
2. Compare that code with the document code printed in full on page 1 and in the table above, every group. A different code means a different report. A matching code is only as good as the page you compared it with.
3. For proof, hand this PDF to anyone with a terminal. The steps below need free tools and no account.

If you do

```
perf-sentinel verify-hash --report report.pdf \
  --verify-binary <perf-sentinel-binary> \
  --expected-issuer <issuer> \
  --expected-identity \
  '<identity>'
# TRUSTED    code, signature and binary all check out
# UNTRUSTED  something was altered, or the signer is not Example SAS's
```

It reads the attachments out of the PDF itself. TRUSTED needs cosign and gh on the PATH, perf-sentinel alone stops at PARTIAL.

What this document cannot prove

This report is not signed, so nothing but the page where Example SAS publishes its codes ties it to Example SAS. The code covers the attached report.json only: a PDF editor can change the printed figures without changing it, so compare the figures with the attached report.